

Confidential

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

발표 대본

표지 · 약 20초

안녕하십니까. 과제 3, 표적 궤적 모의 발표를 시작하겠습니다.

내용은 세 덩어리입니다. 표적을 어떤 구조체로 정의했는지, 그 표적을 시간에 따라 어떻게 움직였는지, 그리고 그 결과가 맞다는 것을 어떻게 확인했는지입니다.

INDEX

표적 궤적 모의
TargetSim

Janghwan Kim (Harley)

01

표적 구조체 설계

상태 벡터와 좌표계 정의
기동을 $G \cdot \text{축} \cdot \text{구간}$ 으로 환원
구조체 관계와 실제 헤더

02

표적 궤적 모의 구현

운동학 모델과 회전행렬 전개
동체 속도 $\rightarrow$ ECEF 속도 (추가 고민)
중점법 적분과 방어적 구현

03

시뮬레이션 결과 및 검증

명세 조건 60s 결과
해석해 대조 · 불변량 보존 · 수렴 차수
기동 시연과 운용 화면

2

발표 대본

목차 · 약 45초

1장은 구조체 설계입니다. 표적 하나의 상태를 어떤 변수로 잡을지, 그리고 "기동"이라는 말을 코드가 다룰 수 있는 값 네 개로 어떻게 바꿨는지를 봅니다.

2장이 구현입니다. 표적 운동을 지배하는 방정식을 먼저 세우고, 거기에 과제에서 추가로 고민해 보라고 한 항목 — 동체 속도를 ECEF 성분으로 바꾸는 방법 — 이 들어갑니다. 0.1초를 전진시키는 수치적분도 여기서 다룹니다.

3장이 결과와 검증입니다. 그림이 그럴듯하게 나왔다는 것만으로는 맞다고 할 수 없어서, 세 가지 방법으로 따로 확인했습니다.

수학은 삼각함수와 행렬 곱셈 정도가 전부이고, 그마저도 필요한 자리에서 다시 설명하겠습니다.

들어가며

과제 요구사항과 발표 구성

「과제 3 내용」의 요구사항을 네 덩어리로 묶고, 각각을 어느 장에서 다루는지 먼저 짚겠습니다.

1 표적 구조체 정의

표적 최대 10, 표적별 기동 최대 30.
위치는 위도 · 경도 · 고도,
속도는 동체 속도 $V_{heading}$ 한 개.
기동은 G · 축 · 시작 · 종료.

01 장

2 표적 상태 갱신

위치와 속도를 ECEF 에서 갱신.
추가 고민 — $V_{heading}$ 을 ECEF
 V_x, V_y, V_z 로 변환하는 방법.

02 장

3 시뮬레이션 조건

$60s \cdot 0.1s$ 간격.
플랫폼 1+ 대함 1+ 대공 1.
제공받은 좌표변환 코드 사용.

03 장

4 결과 제시

궤적을 LLA 로 출력하고 그림으로.
구조체 · 구현 방법 · 결과를
코드와 함께 설명.

03 장

좌표변환과 행렬 연산은 제공받은 코드를 한 줄도 고치지 않고 그대로 사용했습니다. 새로 작성한 것은 TargetSim.c 하나입니다.
발표에 나오는 숫자는 모두 그 Core 를 실제로 빌드해 돌린 결과이며, 원본 CSV 를 함께 제출합니다.

3

발표 대본

도입 · 약 60초

과제 문서의 요구사항을 네 덩어리로 묶었습니다.

첫째가 구조체 정의입니다. 표적은 최대 10개, 표적마다 기동은 최대 30개. 위치는 위도 경도 고도로, 속도는 동체 속도 하나로 받고, 기동에는 G 값과 회전축, 시작과 종료 시각이 들어가야 합니다. 이건 1장에서 다룹니다.

둘째가 상태 갱신인데, ECEF 좌표계에서 하라고 되어 있습니다. 여기에 괄호로 "동체 속도를 ECEF 성분으로 바꾸는 방법을 고민해 보라, 안 해도 된다" 는 항목이 붙어 있습니다. 그런데 이건 안 풀면 구현 자체가 안 되는 부분이라 정면으로 다뤘고, 2장의 한가운데에 놓았습니다.

셋째가 시뮬레이션 조건, 넷째가 결과 제시입니다. 둘 다 3장입니다.

아래 띠에 두 가지를 미리 적어 뒀습니다. 하나, 좌표변환과 행렬 연산은 제공받은 코드를 한 줄도 고치지 않고 그대로 썼습니다. 새로 작성한 것은 TargetSim.c 하나입니다. 둘, 오늘 나오는 숫자는 전부 그 코드를 실제로 빌드해서 돌린 결과이고, 원본 CSV 를 같이 제출했습니다.

들어가며

표적 모의가 들어가는 자리

레이다 데이터처리 소프트웨어를 만들려면, 그 앞단이 내보내는 것과 같은 규격의 입력이 필요합니다.

레이다 데이터처리 체인에서 모의가 대신하는 구간



모의를 쓰는 이유

	실제 해상 시험	표적 모의
반복성	같은 상황 재현 불가	몇 번이든 동일
참값(truth)	기준 장비가 또 필요, 그 값도 오차	입력값이 곧 참값
조건 제어	날씨 · 공역 · 표적 가용성에 종속	G · 침로 · 고도 임의
소요	항공 · 항공기 · 인력 · 기간	60 s 궤적이 5 ms

모의가 만족해야 할 조건

- 규격 일치** 데이터처리 입력과 같은 형식
시각마다 위 · 경 · 고도 — 본 과제는 0.1 s 간격 601 표본
- 물리적 타당성** 불변량이 보존될 것
등속 수평 비행이면 속력과 고도가 변하지 않아야 한다
- 수치적 재현성** 정해진 차수로 수렴할 것
경신 간격 Δt 를 절반으로 줄이면 오차가 1/4로 (2차)

4

발표 대본

도입 · 약 55초

위 그림이 레이더 데이터처리 체인입니다. 안테나가 전파를 받고, 신호처리가 표적 후보를 뽑고, 데이터처리가 그걸 위경고도로 바꿔서 추적 필터로 넘기고, 전시기에 뿌립니다.

저희가 만드는 것은 세 번째 칸부터입니다. 그러면 그 앞의 두 칸이 내보내는 것과 같은 규격의 입력이 있어야 개발을 시작할 수 있습니다. 그 자리를 대신 채우는 게 이번 과제입니다.

왜 실제 시험 대신 모의를 쓰냐면, 표 왼쪽이 그 이유입니다. 배와 비행기를 띄워야 하고, 같은 상황을 두 번 만들 수 없고, 무엇보다 표적이 정확히 어디 있었는지를 알려면 기준 장비가 또 필요한데 그 장비도 오차가 있습니다.

모의는 반대입니다. 제가 넣은 값이 곧 정답입니다. 그래서 알고리즘이 틀렸는지 데이터가 틀렸는지 헛갈릴 일이 없습니다.

오른쪽 세 줄이 그 대신 모의가 지켜야 하는 조건입니다. 규격이 같아야 하고, 물리적으로 말이 돼야 하고, 갱신 간격을 줄이면 오차가 정해진 비율로 줄어야 합니다. 세 번째가 오늘 발표에서 가장 강조하고 싶은 부분입니다.

들어가며

소프트웨어 구성과 인터페이스

연산과 화면을 분리해, 계산 엔진만 따로 빌드해 검증할 수 있게 했습니다.



5

발표 대본

도입 · 약 50초

만든 물건은 두 계층입니다.

아래 연산 계층이 순수 C로 된 TargetSimCore입니다. 이 중에서 CoordinateTransform과 matrixCalcLib는 제공받은 코드 원본이고, 제가 새로 쓴 것은 맨 위 TargetSim.c 하나입니다.

위 표현 계층이 MFC로 만든 화면입니다.

둘 사이에 오가는 것은 오른쪽 구조체 두 개뿐입니다. 설정을 담은 ST_SimConfig를 넣으면 시각마다의 상태인 ST_SimSample이 나옵니다. 공개 함수는 여섯 개고, UI는 Core안을 전혀 모릅니다.

이렇게 나눈 덕을 실제로 봤습니다. 이 발표 자료의 숫자를 뽑을 때 리눅스에서 gcc로 Core만 따로 컴파일해서 돌렸는데, MFC 쪽은 손댈 필요가 없었습니다. 계산과 화면이 붙어 있었다면 못 했을 일입니다.

01 표적 구조체 설계

Target State and Structure Design

- 표적 하나의 상태를 어떤 변수로 잡을 것인가
- "기동" 을 코드가 다룰 수 있는 네 개의 값으로 환원하기
- 설정 (입력) 과 상태 (출력) 를 분리한 이유

6

발표 대본

장절 · 약 25초

1장입니다. 표적 하나를 어떤 숫자들로 적을지 정하는 일입니다.

코드를 짜기 전에 종이에서 결정해야 하고, 한 번 정하면 나중에 바꾸기가 제일 어려운 부분이기도 합니다.

01. 표적 구조체 설계

표적 상태 벡터의 정의

위치 3 + 속도 3 + 자세각 3, 모두 아홉 개 성분으로 표적 하나의 상태가 결정됩니다.

표적 하나의 상태 벡터

$$\mathbf{x}(t) = [\mathbf{p}^e(t)^T \ \mathbf{v}^e(t)^T \ \boldsymbol{\Theta}(t)^T]^T \in \mathbb{R}^9$$

$\mathbf{p}^e = [x \ y \ z]^T$	ECEF 위치	m	3
$\mathbf{v}^e = [v_x \ v_y \ v_z]^T$	ECEF 속도	m/s	3
$\boldsymbol{\Theta} = [\phi \ \theta \ \psi]^T$	자세각 (Roll, Pitch, Yaw)	rad	3

위치와 속도는 ECEF 에서 경신하고, LLA 는 출력 다음 스텝용으로 함께 보관한다.

명세가 주는 입력 → 초기 상태

위도 · 경도 · 고도 (ϕ_0, λ_0, h_0) $\mathbf{p}_0 = f_{lla \rightarrow ecef}$	자세각 $(\phi_0, \theta_0, \psi_0)$ (그대로) $\boldsymbol{\Theta}_0$	동체 속력 1개 V $\mathbf{v}_0' = C_o^c C_o^b [V \ 0 \ 0]^T$ → 세 번째 줄이 이 과제의 추가 고민 항목
---	--	--

7

발표 대본

구조체 · 약 60초

표적 하나를 다음 순간으로 옮기려면 지금 무엇을 알아야 하는가. 이걸 상태 벡터라고 부릅니다.

셋으로 묶었습니다. ECEF 위치 세 개, ECEF 속도 세 개, 자세각 세 개. 합쳐서 아홉 개입니다.

위치와 속도를 ECEF 로 잡은 것은 명세 조건이고, 왜 그래야 하는지는 2장에서 따로 보겠습니다. 자세각은 Roll, Pitch, Yaw 세 개입니다.

아래가 명세가 주는 입력이 이 상태로 어떻게 들어가는지입니다.

위도 경도 고도는 좌표변환 함수 한 번으로 ECEF 위치가 됩니다. 자세각은 그대로 들어갑니다.

문제는 세 번째 줄입니다. 명세는 속력 하나만 줍니다. 그런데 상태에는 속도 세 성분이 필요합니다. 이 한 줄을 채우는 게 과제에서 추가로 고민해 보라고 한 항목이고, 2장에서 자세히 다루겠습니다.

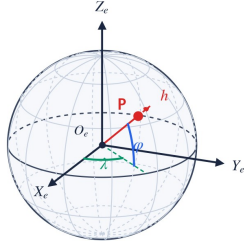
01. 표적 구조체 설계

쓰는 좌표계 — ECEF · LLA · NED · Body

Part 2 에서 정리한 좌표계 중 표적 모의에 실제로 필요한 것은 이 넷입니다.

ECEF — 지구중심 지구고정 좌표계

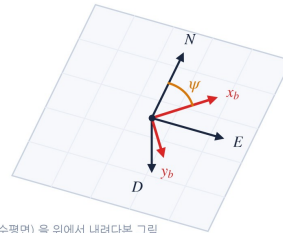
원점 지구중심, Z_e 자전축, X_e 본초자오선. 지구와 함께 돈다.



같은 점 P 를 각도로 적은 것이 LLA (ϕ, λ, h) — 입·출력 전용

NED · Body — 국지 좌표계

원점은 표적 자신. N 북 · E 동 · D 자심방향, x_b 는 기수 방향.



국지 접평면 (수평면) 을 위에서 내려다본 그림
수평 비행이면 $\phi = \theta = 0$ 이라 x_b 와 N 사이 각이 곧 ψ 다



8

발표 대본

구조체 · 약 65초

좌표계는 Part 2 에서 여섯 개를 봤는데, 표적을 움직이는 데 실제로 쓰는 것은 넷입니다.

왼쪽이 ECEF 입니다. 지구 중심이 원점이고, Z 축이 자전축, X 축이 본초자오선 방향입니다. 지구에 박혀 있어서 지구와 같이 돕니다. 단위는 미터입니다.

같은 점 P 를 각도로 적으면 위도 경도 고도, LLA 가 됩니다. 그림에서 파란 호가 위도, 초록 호가 경도, 빨간 점선이 고도입니다. 같은 점을 다르게 적은 것뿐이라 서로 언제든지 바꿀 수 있습니다.

오른쪽이 표적이 있는 자리를 확대한 것입니다. 표적 발밑에 접평면을 하나 깔고, 북쪽을 N, 동쪽을 E, 지구 중심 쪽을 D 로 잡은 것이 NED 입니다.

그 위에 빨간 축이 동체 좌표계입니다. x_b 가 기수 방향입니다. 수평 비행이면 Roll 과 Pitch 가 0 이라, 기수 축과 북쪽 사이의 각이 곧 Yaw 입니다. 주향색 호가 그 각입니다.

아래 띠가 넷을 잇는 순서입니다. 사람은 LLA 로 말하고, 방향은 NED 에서 풀고, 더하기는 ECEF 에서 합니다. 이번 과제는 이 화살표를 거꾸로 타고 갑니다.

01. 표적 구조체 설계

자세각 정의와 회전 순서

Roll · Pitch · Yaw 가 각각 어느 축 둘레의 회전인지, 그리고 곱하는 순서를 고정합니다.



9

발표 대본

구조체 · 약 55초

자세각 셋이 각각 어느 축 둘레의 회전인지 정리하겠습니다.

왼쪽 Roll 은 기수 방향 축, 그러니까 x_b 둘레입니다. 정면에서 본 그림이고 날개가 기웁니다. 회전축이 화면 밖으로 나오니까 가운데를 점으로 찍었습니다.

가운데 Pitch 는 오른쪽 날개 축, y_b 둘레입니다. 옆에서 본 그림이고 기수가 들립니다. 고도가 바뀌는 회전입니다.

오른쪽 Yaw 는 아래 방향 축, z_b 둘레입니다. 위에서 본 그림이고 기수가 좌우로 돕니다. 수평 선회가 이 축입니다. Pitch 와 Yaw 는 축이 화면 안으로 들어가서 엑스 표시로 찍었습니다.

세 그림 모두 회색 파선이 회전 전 기준선입니다.

아래가 중요합니다. 동체에서 NED 로 가는 회전행렬은 이 세 회전의 곱인데, 순서가 정해져 있습니다. Yaw 먼저, 그다음 Pitch, 마지막이 Roll 입니다. 순서를 바꾸면 다른 자세가 됩니다.

그리고 빨간 줄. 제공받은 $f_Trans_Body_To_Ned$ 함수는 인자를 Roll, Yaw, Pitch 순서로 받습니다. 구조체 필드 순서인 Roll, Pitch, Yaw 와 다릅니다. 여기서 한 번 틀려서 고생했기 때문에 호출하는 자리마다 주석을 남겨 뒀습니다.

01. 표적 구조체 설계

기동을 네 개의 값으로

"12 초부터 16 초까지 오른쪽으로 8 G" 라는 지시를 구조체 한 줄로 옮깁니다.

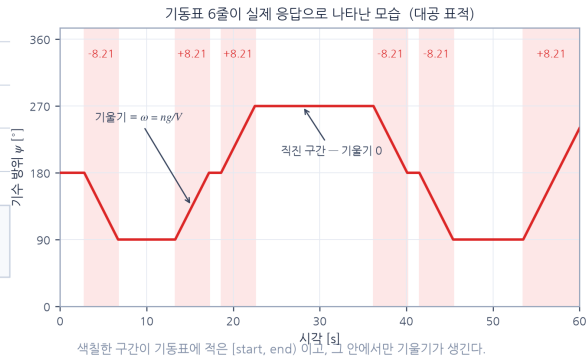
기동 1건 = 값 네 개

enTurnType 0 없음 / 1 Roll / 2 Yaw / 3 Pitch	EN_TurnType
gravityValue 크기는 세기, 부호는 회전 방향	FLOAT64 [g]
startTime 이 시각부터 (포함)	FLOAT64 [s]
endTime 이 시각까지 (제외)	FLOAT64 [s]

구간 규칙

$t \in [t_{start}, t_{end})$ 끝과 다음 시작이 같아도 겹침이 아니다
같은 표적에서 두 구간이 실제로 겹치면 TGT_ERR_MANEUVER_OVERLAP

표적 하나에 최대 30줄, 걸린 구간에서만 자세각이 돌고, 나머지 시간은 직진이다.
기동 지시는 보통 "위도 32.1 에서 좌 90도" 처럼 좌표로 온다. 구조체에는 시각 칸만 있으므로 Core 를 실제로 돌려 그 좌표에 닿는 시각을 찾아 start / end 로 옮겼다.



10

발표 대본

구조체 · 약 60초

다음이 기동입니다. "12초부터 16초까지 오른쪽으로 8G 로 꺾어라" 라는 말을 구조체 한 줄로 바꿔야 합니다.

왼쪽 위가 그 네 개입니다. 회전축, G 값, 시작 시각, 종료 시각. 축은 명세가 정한 그대로 0이 없음, 1이 Roll, 2가 Yaw, 3이 Pitch 입니다. G 값은 크기가 세기, 부호가 방향입니다.

그 아래 구간 규칙을 하나 정했습니다. 시작은 포함하고 끝은 포함하지 않습니다. 그래야 한 기동이 16초에 끝나고 다음이 16초에 시작해도 겹침으로 잡히지 않고, 연속 기동을 한 줄씩 이어 붙일 수 있습니다. 실제로 겹치는 경우에는 계산을 시작하기 전에 오류 코드로 돌려보냅니다.

오른쪽 그림이 그 기동표가 실제 결과로 어떻게 나타나는지입니다. 세로축이 기수 방위각이고, 분홍색 구간이 기동 표에 적은 구간입니다.

색칠한 데서만 선이 기울고, 나머지는 평평합니다. 그리고 기울어진 정도가 각속도인데, 그 값이 다음 장의 식에서 나옵니다.

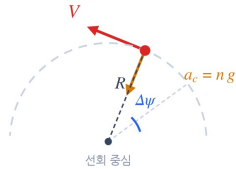
한 가지 덧붙이면, 기동 지시는 보통 "위도 32.1 에서 좌 90도" 처럼 좌표로 옵니다. 구조체에는 시각 칸밖에 없어서, Core 를 실제로 돌려 그 좌표에 닿는 시각을 찾아 옮기는 작업을 따로 했습니다.

01. 표적 구조체 설계

G 값이 각속도와 선회 반경을 정한다

각속도를 직접 받지 않고 G 로 받는 이유는, G 가 조종사와 기체가 실제로 견디는 한계이기 때문입니다.

수평 선회의 기하



등속 원운동이므로 속도 V 는 접선 방향,
가속도 a_c 는 언제나 중심을 향한다.
이 그림 한 장에서 오른쪽 세 줄이 모두 나온다.

G 값에서 각속도와 선회 반경이 나온다

$$a_c = \frac{V^2}{R} = V\omega$$

원운동의 구심가속도

$$a_c = n g, \quad g = 9.80665 \text{ m/s}^2$$

기동 세기를 중력가속도의 n 배로 적는다

$$\omega = \frac{n g}{V}, \quad R = \frac{V^2}{n g}$$

G 값 하나가 회전 속도와 반경을 함께 정한다

코드에서는 한 줄이다 —

```
omega = (gravityValue * G_FORCE) / headingSpeed;
```

본 과제 두 표적에 G 를 걸면

n [G]	대함 V = 30 m/s	대공 V = 200 m/s
1	18.7 °/s R = 91.8 m	2.8 °/s R = 4079 m
2	37.5 °/s R = 45.9 m	5.6 °/s R = 2039 m
4	74.9 °/s R = 22.9 m	11.2 °/s R = 1020 m
8	149.8 °/s R = 11.5 m	22.5 °/s R = 510 m

같은 G 라도 빠른 표적은 덜 꺾인다 — 분모에 속력이 있기 때문이다.

11

발표 대본

구조체 · 약 65초

왜 각속도를 직접 받지 않고 G 로 받을까요.

G 가 실제 한계값이기 때문입니다. 전투기는 9G 근처가 한계이고 조종사도 그 근처에서 의식을 잃습니다. 시나리오를 만드는 사람은 "이 표적이 낼 수 있는 G 가 얼마인가" 로 생각하지, "초당 몇 도" 로 생각하지 않습니다.

왼쪽이 수평 선회의 기하입니다. 속도는 원의 접선 방향이고, 가속도는 항상 중심을 향합니다. 이것 구심가속도라고 합니다.

오른쪽이 유도입니다. 세 줄입니다.

첫 줄, 원운동의 구심가속도는 속력 제곱 나누기 반경이고, 이건 속력 곱하기 각속도와 같습니다.

둘째 줄, 그 구심가속도를 중력가속도의 n 배로 적자는 게 G 값입니다.

셋째 줄, 두 줄을 합치면 각속도는 n 곱하기 9.80665 나누기 V , 선회 반경은 V 제곱 나누기 n 곱하기 9.80665 입니다. G 값 하나가 회전 속도와 반경을 동시에 정합니다.

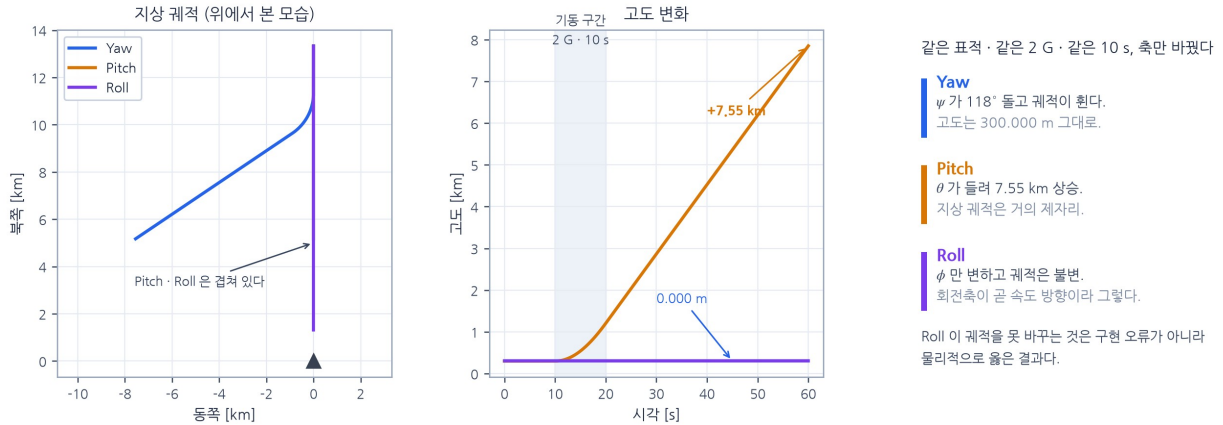
코드에서는 그 밑의 한 줄이 전부입니다.

아래 표가 이번 과제 두 표적에 실제로 G 를 걸었을 때 값입니다. 여기서 읽으실 것은, 같은 G 라도 빠른 표적이 훨씬 덜 꺾인다는 점입니다. 분모에 속력이 있으니까요. 초속 30미터 대함은 1G 로도 초당 18.7도를 도는데, 초속 200미터 대공은 2.8도밖에 못 돕니다. 반경도 92미터 대 4킬로미터로 차이가 큼니다.

01. 표적 구조체 설계

회전축 세 가지의 응답 — 실측

같은 표적에 같은 2 G 를 10 초씩, 축만 바꿔 걸어 본 결과입니다.



12

발표 대본

구조체 · 약 60초

축이 셋이라고 말로만 하면 감이 안 오니까 실제로 돌려 봤습니다.

같은 대공 표적, 같은 2G, 같은 10초입니다. 축만 바꿨습니다.

왼쪽이 위에서 본 지상 궤적입니다. 파란색 Yaw 는 기수가 옆으로 돌아 궤적이 크게 휘었습니다. 118도 돌았습니다.

가운데가 고도입니다. 주황색 Pitch 가 10초 동안 7.55킬로미터를 올라갔습니다. 대신 위에서 보면 거의 제자리입니다. 파란 Yaw 는 고도가 300.000미터 그대로입니다.

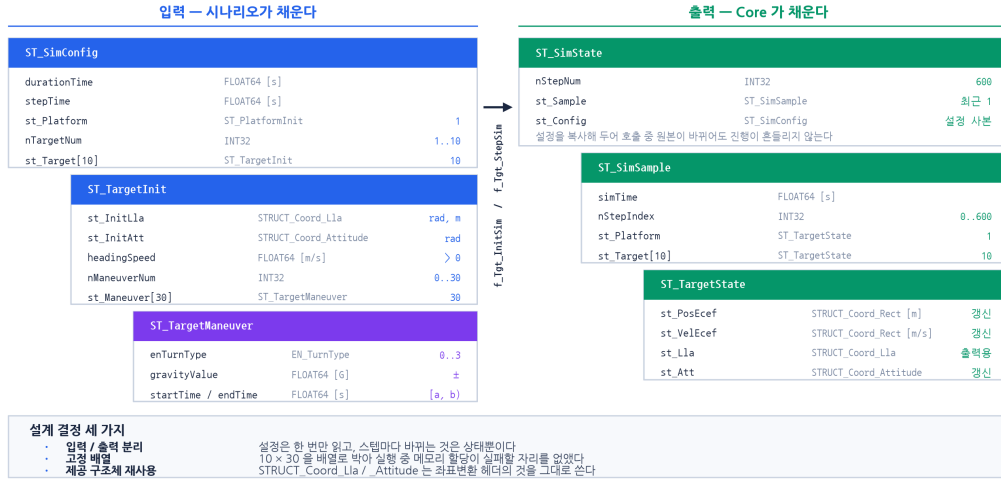
보라색 Roll 이 재미있습니다. 궤적도 고도도 전혀 안 변했습니다. 처음에는 구현이 틀린 줄 알고 한참 봤습니다.

그런데 물리적으로 맞는 결과였습니다. Roll 축이 곧 속도 방향 그 자체이기 때문입니다. 자기가 가는 방향을 축으로 삼아 몸통만 돌리는 거라, 방향이 바뀔 수가 없습니다.

실제 비행기도 Roll 만으로는 방향을 못 바꿉니다. Roll 로 기울인 다음 Pitch 를 당겨야 선회가 됩니다. 저희 모델은 Roll 과 Pitch 를 독립적으로 적분하니까 그 결합이 없고, 그래서 Roll 이 궤적을 바꾸지 않습니다. 이건 모델의 한계이기도 해서 뒤에 다시 언급하겠습니다.

01. 표적 구조체 설계

구조체 관계와 설계 결정



13

발표 대본

구조체 · 약 55초

지금까지 이야기한 것을 구조체로 그리면 이렇게 됩니다.

왼쪽이 입력입니다. 맨 위 ST_SimConfig 가 시뮬레이션 시간과 갱신 간격, 표적 수를 들고, 그 안에 플랫폼 하나와 표적 배열이 들어갑니다. 표적마다 기동 배열이 또 들어갑니다. 10개와 30개는 배열 크기로 박아 뒀습니다.

오른쪽이 출력입니다. ST_SimState 가 진행 상태를 들고, 그 안의 ST_SimSample 이 "이 시각 전원의 상태" 한 장입니다. 그 안에 객체마다 ST_TargetState 가 있고, 거기에 ECEF 위치, ECEF 속도, LLA, 자세각이 들어갑니다.

각 필드 옆에 타입과 단위, 개수를 같이 적어 뒀습니다. 단위를 문서가 아니라 구조체 옆에 적어 두는 게 나중에 헛갈리지 않는 방법이라고 봤습니다.

아래 세 가지가 설계 결정입니다.

입력과 출력을 완전히 갈랐습니다. 설정은 처음 한 번만 읽고, 매 스텝 바뀌는 것은 상태뿐입니다. 섞어 두면 "이 값이 내가 넣은 값인지 계산된 값인지" 가 헛갈립니다.

고정 배열을 썼습니다. 10 곱하기 30 이면 구조체가 10킬로바이트 정도인데, 요즘 기준으로 작은 값이고 대신 실행 중에 메모리 할당이 실패할 자리가 없습니다.

그리고 위치와 자세 구조체는 제공받은 좌표변환 헤더에 이미 있던 것을 그대로 썼습니다. 새로 만들면 변환할 때마다 복사해 넣어야 하니까요.

01. 표적 구조체 설계

코드로 보는 구조체 ① — 입력

TargetSimCore/include/TargetSim.h

```
// 기동 회전축 . 값은 과제 명세를 그대로 따른다 .
typedef enum
{
    TGT_TURN_NONE = 0,   TGT_TURN_ROLL  = 1,
    TGT_TURN_YAW  = 2,   TGT_TURN_PITCH = 3
} EN_TurnType;

// 기동 1건 . gravityValue 의 부호가 회전 방향이며
// 동체축 오른손 법칙을 따른다 .
typedef struct
{
    EN_TurnType  enTurnType;
    FLOAT64      gravityValue; // [g] 중력가속도 배수
    FLOAT64      startTime;    // [s]
    FLOAT64      endTime;      // [s]
} ST_TargetManeuver;

// 위치는 위도 / 경도 / 고도 , 속도는 동체 x 축 방향 크기 .
typedef struct
{
    STRUCT_Coord_Lla    st_InitLla; // Lat, Lon [rad] / Alt [m]
    STRUCT_Coord_Attitude st_InitAtt; // Roll, Pitch, Yaw [rad]
    FLOAT64              headingSpeed; // V_heading [m/s]
    INT32                nManeuverNum;
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

구간은 [시작, 종료)

한 기동의 끝과 다음 기동의 시작이 같아도 겹침으로 보지 않는다 . 경계를 열어 두어야 연속 기동을 한 줄씩 이어 붙일 수 있다 .

겹치면 계산 전에 거부

같은 표적에서 두 구간이 실제로 겹치면 어느 쪽을 따를지 정의되지 않는다 . TGT_ERR_MANEUVER_OVERLAP 을 돌려준다 .

10 × 30 은 고정 배열

구조체 하나가 약 10 KB. 동적 할당이 없어 실행 중 메모리 실패가 생길 자리가 없다 .

14

발표 대본

코드 · 약 60초

실제 헤더 파일입니다.

위가 회전축 enum 입니다. 값을 0, 1, 2, 3 으로 명세와 똑같이 맞췄습니다. 이렇게 해 두면 화면에서 사용자가 고른 번호를 그대로 넣을 수 있어서 변환표가 필요 없습니다.

가운데가 기동 한 건입니다. 앞에서 본 네 개가 그대로 들어가 있습니다.

아래가 표적 초기 설정입니다. 위치가 STRUCT_Coord_Lla, 자세가 STRUCT_Coord_Attitude 인데 방금 말씀드린 대로 제공받은 구조체입니다. 단위가 라디안인 것도 그 헤더를 따랐습니다. 그 아래 headingSpeed 가 속력 하나입니다.

오른쪽 메모 셋만 짤겠습니다.

구간을 반열린 구간으로 둔 이유는 앞에서 말씀드렸습니다.

겹치면 계산 전에 거부합니다. 시작해 놓고 중간에 이상한 값이 나오는 것보다, 시작하기 전에 못 하겠다고 하는 쪽이 낫다고 봤습니다.

10 곱하기 30 은 고정 배열입니다.

01. 표적 구조체 설계

코드로 보는 구조체 ② — 출력

TargetSimCore/include/TargetSim.h

```
// 한 시각의 상태 . 위치와 속도는 ECEF 에서 갱신하고 ,
// LLA 는 출력과 다음 스텝의 NED 기준으로 함께 보관한다 .
typedef struct
{
    FLOAT64      simTime;      // [s]
    STRUCT_Coord_Rect st_PosEcef; // [m]
    STRUCT_Coord_Rect st_VelEcef; // [m/s]
    STRUCT_Coord_Lla st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 . 표적 상태 , 호출자는 스텝마다
// 이 구조체를 복사해 보관한다 .
typedef struct
{
    FLOAT64      simTime;
    INT32      nStepIndex;
    INT32      nTargetNum;
    ST_TargetState st_Platform;
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

ECEF 와 LLA 를 함께 두는 이유

- 갱신은 ECEF 에서 해야 곡률 향이 사라진다
 - 출력 · 화면 · CSV 는 LLA 라야 읽힌다
 - 다음 스텝의 속도를 만들 때도 그 지점의 위도 · 경도가 필요하다
- 스텝당 한 번만 변환해 두면 세 용도를 모두 덮는다.

플랫폼도 같은 구조체

플랫폼은 기동이 없는 표적일 뿐이다.
같은 ST_TargetState 를 쓰고 각속도만 0 으로
넣어 같은 적분 경로를 통과시킨다.

속력으로 나누는 연산은 기동 조회 안에만 있어,
정지 플랫폼 ($V = 0$) 도 안전하다.

15

발표 대본

코드 · 약 55초

출력 쪽입니다.

위가 객체 하나의 상태입니다. 눈여겨보실 것은 파란 두 줄, ECEF 위치와 속도를 들고 있으면서 그 아래 LLA 도 같이 들고 있다는 점입니다.

값이 중복되는 것처럼 보이는데 이유가 셋입니다.

하나, 갱신은 ECEF 에서 해야 지구 곡률 향이 사라집니다. 둘, 출력과 화면과 CSV 는 LLA 라야 읽힙니다. 셋, 이게 덜 알려진 부분인데 다음 스텝의 속도를 만들 때도 그 지점의 위도와 경도가 필요합니다. NED 축을 세우려면 내가 지금 지구 어디에 있는지를 알아야 하니까요.

세 군데에서 쓰는데 스텝마다 한 번만 변환해 두면 되니까, 들고 있는 쪽이 이득입니다.

아래가 한 시각의 전원 상태입니다. 플랫폼이 따로 한 칸, 표적이 배열로 들어갑니다.

플랫폼도 같은 ST_TargetState 를 씁니다. 플랫폼은 기동이 없는 표적일 뿐이라, 각속도만 0 으로 넣고 똑같은 함수를 통과시킵니다. 코드가 두 벌이 안 됩니다. 그리고 속력으로 나누는 연산이 기동 조회 안에만 있어서, 속력이 0 인 정지 플랫폼도 안전합니다.

02 표적 궤적 모의 구현

Trajectory Propagation

- 표적 운동을 지배하는 방정식과 그 가정
- 동체 속도 한 개를 ECEF 속도 세 성분으로 — 과제의 추가 고민 항목
- 0.1 초를 전진시키는 방법과 오차 차수

16

발표 대본

장절 · 약 25초

2장입니다. 구조체에 담긴 값을 실제로 움직이는 부분입니다.

여기가 과제의 핵심이고, 추가로 고민해 보라고 한 항목도 이 안에 있습니다.

02. 표적 궤적 모의 구현

표적 운동을 지배하는 세 줄

이 세 줄이 곧 구현이고, 아래 세 가지 가정을 명시적으로 두고 있습니다.

표적 운동을 지배하는 세 줄

$$\dot{\mathbf{p}}^e = \mathbf{v}^e$$

위치의 시간 변화율이 곧 속도다

$$\mathbf{v}^e = \mathbf{C}_n^e(\phi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

속도는 기수 방향 크기 V 를 두 번 돌려 만든다

$$\dot{\boldsymbol{\theta}} = \boldsymbol{\omega}(t), \quad \omega_i = n_i g / V$$

자세각은 기동표가 지시하는 각속도로 돈다 (기동이 걸린 구간에서만 0 이 아니다)

이 모델이 전제하는 것

운동학 모델

힘이 아니라 속도 · 각속도를 직접 준다.

중력 · 추력 · 항력을 풀지 않으므로 탄도 표적에는 맞지 않는다.

속력 일정

회전만 하므로 $|v|$ 는 변하지 않는다.

가속 · 감속이 필요하면 V 를 시간 함수로 바꿔야 한다.

오일러각 적분

자세각을 직접 적분한다.

$\theta \rightarrow \pm 90^\circ$ 에서 짐벌락 — 사원수로 바꾸면 풀린다.

17

발표 대본

구현 · 약 60초

구현을 시작하기 전에 방정식부터 세웠습니다. 세 줄입니다.

첫 줄. 위치의 시간 변화율이 곧 속도다. 이걸 정의입니다.

둘째 줄. 속도는 기수 방향 크기 V 를 두 번 돌려 만든다. 회전행렬 두 개가 곱해져 있는데, 오른쪽이 동체에서 NED 로, 왼쪽이 NED 에서 ECEF 로 가는 행렬입니다. 이 줄이 뒤에 나올 추가 고민 항목의 답입니다.

셋째 줄. 자세각은 기동표가 지시하는 각속도로 돈다. 그 각속도가 앞에서 본 n 곱하기 g 나누기 V 이고, 기동이 걸린 구간에서만 0 이 아닙니다.

아래 세 칸이 이 모델이 전제하는 것입니다. 이걸 명시적으로 적어 두는 게 중요하다고 봤습니다.

첫째, 운동학 모델입니다. 힘이 아니라 속도와 각속도를 직접 줍니다. 중력이나 추력, 항력을 풀지 않으니까 탄도 표적에는 맞지 않습니다.

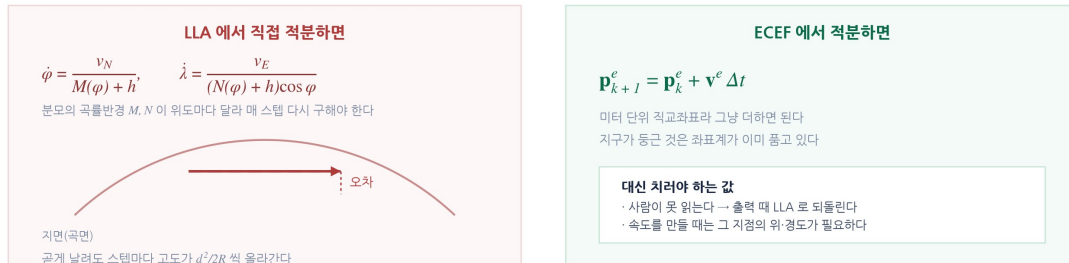
둘째, 속력이 일정합니다. 회전만 하니까 크기가 안 변합니다. 가속하는 표적이 필요하면 V 를 시간 함수로 바꿔야 합니다.

셋째, 오일러각을 직접 적분합니다. 그래서 Pitch 가 90도 근처로 가면 짐벌락이 생깁니다. 사원수로 바꾸면 풀리는데, 이번 과제 범위를 넘는다고 보고 남겨 뒀습니다.

02. 표적 궤적 모의 구현

왜 ECEF 에서 적분하는가

명세가 정한 조건이지만, 왜 그래야 하는지는 오차 구조를 보면 분명합니다.



본 과제 조건에서 실제로 얼마인가

한 스텝 이동거리 $d = V\Delta t$

$200 \times 0.1 = 20 \text{ m}$

스텝당 고도 편차 $d^2/2R$

$31.36 \text{ } \mu\text{m}$

600 스텝 누적 (보정 없이)

18.81 mm

중점법으로 보정한 실속

고도 300.000000 m 고정

값 자체는 작지만, 이런 항이 남아 있으면 갱신 간격을 줄여도 오차가 선형으로만 줄어든다 → 2차 수렴이 깨진다.

18

발표 대본

구현 · 약 60초

명세가 ECEF 에서 갱신하라고 정해 놔는데, 왜 그런지 직접 계산해 보면 이유가 분명합니다.

왼쪽이 LLA 에서 바로 적분하는 경우입니다. 위도의 변화율을 쓰려면 분모에 곡률반경이 들어갑니다. 그런데 그 곡률반경이 위도마다 다릅니다. 그러니까 매 스텝 다시 구해야 합니다.

그것만이면 계산이 좀 늘어나는 정도인데, 진짜 문제는 그 아래입니다. 곧게 날려도 스텝마다 고도가 조금씩 올라갑니다. 지면이 곡면인데 표적은 직선으로 나아가려고 하니까, 그 차이가 매번 쌓입니다.

오른쪽이 ECEF 입니다. 미터 단위 직교좌표라 새 위치는 옛 위치에 속도 곱하기 델타티를 그냥 더하면 됩니다. 지구가 둥근 것은 좌표계가 이미 품고 있습니다.

대신 치려야 하는 값이 있습니다. 사람이 못 읽으니까 출력할 때 되돌려야 하고, 속도를 만들 때는 그 지점의 위경도가 필요합니다. 그래서 앞에서 LLA 를 같이 들고 있다고 말씀드린 겁니다.

아래가 실제 숫자입니다. 대공 표적은 0.1초에 20미터를 갑니다. 그러면 스텝당 고도 편차가 31마이크로미터, 600스텝이면 18.8밀리미터입니다.

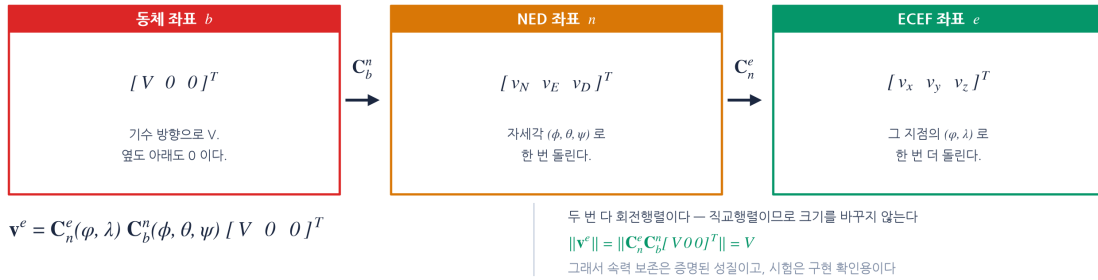
값 자체는 작습니다. 그런데 중요한 건 크기가 아니라 성질입니다. 이런 항이 남아 있으면 갱신 간격을 줄여도 오차가 선형으로만 줄어듭니다. 2차 수렴이 깨집니다. 마지막 줄이 실제 결과인데, 중점법으로 보정한 뒤에는 고도가 소수점 여섯째 자리까지 고정입니다.

02. 표적 궤적 모의 구현

동체 속도를 ECEF 속도로 — 과제의 추가 고민

" 표적 동체 속도 값 ($V_heading$) 을 ECEF 좌표계 V_x, V_y, V_z 성분으로 변환하는 방법 " 에 대한 답입니다 .

동체 속도를 ECEF 속도로 — 회전 두 번



구현에서 주의한 두 가지

① 평행이동 제거

f_Trans_Ned_To_Ecef 는 회전에 더해 기준점만큼 평행이동까지 한다. 속도는 위치가 아니라 방향으로 기준점 거리에 (0, 0, 0) 을 넣어 회전만 남겼다.

② 인자 순서

f_Trans_Body_To_Ned 는 (Roll, Yaw, Pitch) 순서로 받는다. 자세각 구조체 필드 순서(Roll, Pitch, Yaw)와 달라 호출부에 주석을 남겼다.

19

발표 대본

구현 · 약 65초

과제에서 추가로 고민해 보라고 한 항목입니다. 동체 속도 $V_heading$ 을 ECEF 의 V_x, V_y, V_z 로 어떻게 바꿀 것인가.

결론부터 말씀드리면 회전 두 번입니다.

출발은 동체 좌표입니다. 기수 방향이 x 축이니까 속도는 그냥 $V, 0, 0$ 입니다. 옆도 0, 아래도 0. 이게 제일 쉬운 형태이고, 속도를 크기 하나로 받은 덕입니다.

첫 번째 회전이 동체에서 NED 로 가는 것입니다. 자세각으로 돌리면 북쪽 성분, 동쪽 성분, 아래 성분이 나옵니다. Yaw 가 180도면 북쪽 성분이 마이너스 V , 나머지가 0 이 나오겠죠. 남쪽으로 간다는 뜻입니다.

두 번째 회전이 NED 에서 ECEF 로 가는 것입니다. 여기에 그 지점의 위도와 경도가 들어갑니다. 같은 "북쪽" 이라도 제주 앞바다에서의 북쪽과 뉴욕에서의 북쪽은 ECEF 에서 전혀 다른 방향이니까요.

가운데 아래 식이 그 둘을 합친 것입니다.

그 오른쪽이 이 방식의 이점입니다. 두 번 다 회전행렬이고, 회전행렬은 직교행렬이라 벡터의 크기를 바꾸지 않습니다. 그래서 결과 속도의 크기는 처음 넣은 V 와 정확히 같습니다.

이건 증명된 성질이지 시험으로 알아내는 게 아닙니다. 다만 구현이 그 성질을 지키는지는 확인해야 해서, 뒤에서 601개 표본의 속력을 재 본 결과를 보여 드리겠습니다.

아래 두 줄이 구현할 때 주의한 부분인데, 다음다음 장에서 코드로 보겠습니다.

02. 표적 궤적 모의 구현

회전행렬의 원소

두 행렬을 펼치면 이렇게 됩니다. 위치에는 평행이동이 붙고, 속도에는 붙지 않습니다.

① 동체 → NED

$$\mathbf{C}_b^n(\phi, \theta, \psi)$$

$$\begin{bmatrix} c\psi c\theta & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix}$$

ϕ Roll θ Pitch ψ Yaw ($c = \cos$, $s = \sin$)

세 축 회전 $\mathbf{R}_x(\psi)\mathbf{R}_y(\theta)\mathbf{R}_z(\phi)$ 를 곱해 펼친 것이다.

② NED → ECEF

$$\mathbf{C}_n^e(L, \lambda)$$

$$\begin{bmatrix} -sL c\lambda & -s\lambda & -cL c\lambda \\ -sL s\lambda & c\lambda & -cL s\lambda \\ cL & 0 & -sL \end{bmatrix}$$

L 위도 λ 경도

표적이 움직이면 이 행렬도 매 스텝 다시 만들어야 한다.

두 행렬을 쓰는 방법 — 위치는 평행이동까지, 속도는 회전만

$$\mathbf{p}^e = \mathbf{C}_n^e \mathbf{p}^n + \mathbf{p}_{ref}^e$$

위치를 옮길 때는 기준점을 더한다

`f_Trans_Ned_To_Ecef(..., Pf_X, Pf_Y, Pf_Z, ...)`

$$\mathbf{v}^e = \mathbf{C}_n^e \mathbf{v}^n$$

속도는 방향뿐이라 기준점을 0 으로 준다

`f_Trans_Ned_To_Ecef(..., 0.0, 0.0, 0.0, ...)`

20

발표 대본

구현 · 약 50초

두 행렬을 펼치면 이렇게 됩니다. 외우실 필요는 없고 구조만 보시면 됩니다.

왼쪽이 동체에서 NED 로 가는 행렬입니다. Roll, Pitch, Yaw 세 각의 사인 코사인이 섞여 있습니다. 세 축 회전을 순서대로 곱해서 펼친 결과이고, c 는 코사인, s 는 사인입니다.

오른쪽이 NED 에서 ECEF 로 가는 행렬입니다. 이건 훨씬 단순해서 위도와 경도만 들어갑니다. 표적이 움직이면 위 경도가 바뀌니까, 이 행렬도 매 스텝 다시 만들어야 합니다.

아래가 두 행렬을 실제로 쓰는 방법인데, 여기가 제가 한 번 틀렸던 자리입니다.

왼쪽. 위치를 옮길 때는 회전한 다음 기준점을 더합니다. 원점이 다르니까요.

오른쪽. 속도는 방향뿐이라 기준점을 더하면 안 됩니다. 그런데 제공받은 함수는 위치를 옮기라고 만든 거라 평행이동이 들어 있습니다. 그래서 기준점 자리에 0, 0, 0 을 넣어서 회전만 남겼습니다.

02. 표적 궤적 모의 구현

코드 ③ — 동체 속도를 ECEF 속도로

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번, 위경도로 한 번 돌려 ECEF 속도로 만든다.
// 회전만 거치므로 속력은 그대로 보존된다.
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다.
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
    st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다. 방향만 옮길 때는 기준점을 원점으로 준다.
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
    0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

기준점을 0 으로 주는 이유

f_Trans_Ned_To_Ecef 는 회전에 더해
기준점만큼 평행이동까지 한다. 속도는
"위치"가 아니라 "방향"이라 빼야 한다.

인자 순서를 주석에 남긴 이유

제공받은 함수가 Roll, Yaw, Pitch 순서로
받는다. 자세각 구조체 필드 순서와 달라
한 번 틀렸던 자리다.

속력 보존은 성질, 시험은 확인

회전행렬은 직교행렬이라 크기를 바꾸지
않는다. 601 표본에서 속력이 소수점
12 자리까지 입력값과 일치했다.

21

발표 대본

코드 · 약 55초

방금 설명한 게 코드로는 실행문 두 줄입니다.

첫 줄이 동체에서 NED 입니다. headingSpeed 를 첫 인자로 주고 나머지 둘을 0 으로 줍니다. 이게 "기수 방향으로 V" 라는 뜻입니다.

둘째 줄이 NED 에서 ECEF 인데, 기준점 자리에 0 이 세 개 들어가 있는 게 보이실 겁니다. 방금 말씀드린 그 부분입니다.

아래 카드 셋이 이 함수에서 신경 쓴 것들입니다.

왼쪽이 기준점 이야기입니다.

가운데가 인자 순서입니다. 제공받은 함수가 Roll, Yaw, Pitch 순서로 받는데 구조체 필드는 Roll, Pitch, Yaw 순서입니다. 컴파일도 되고 실행도 되는데 결과만 틀리는 종류의 실수라, 주석을 남겨 뒀습니다.

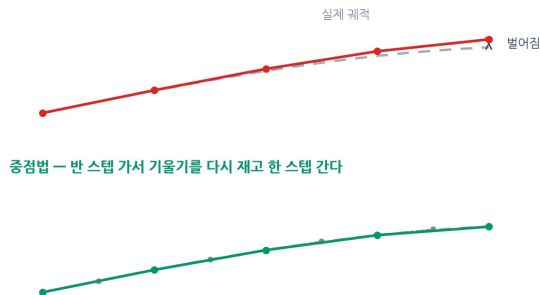
오른쪽이 속력 보존 확인입니다. 601개 표본에서 속력이 소수점 열두 자리까지 입력값과 같았습니다.

02. 표적 궤적 모의 구현

0.1 초를 어떻게 전진시킬 것인가

속도가 계속 바뀌는 구간에서 어느 지점의 기울기를 쓰느냐가 오차 차수를 가릅니다.

오일러법 — 출발할 때의 기울기로 한 스텝 간다



중점법 — 반 스텝 가서 기울기를 다시 재고 한 스텝 간다

굵은 길을 직선 토막으로 잇는 것은 같다. 다른 것은 어느 지점의 기울기를 쓰느냐다 — 중점의 기울기가 구간 평균에 훨씬 가깝다.

두 방법의 식과 오차 차수

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f(t_k, \mathbf{x}_k)$$

국소 $O(\Delta t^2)$ · 전역 $O(\Delta t)$ — 1차

$$\mathbf{k}_1 = f(t_k, \mathbf{x}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f\left(t_k + \frac{\Delta t}{2}, \mathbf{x}_k + \frac{\Delta t}{2} \mathbf{k}_1\right)$$

국소 $O(\Delta t^3)$ · 전역 $O(\Delta t^2)$ — 2차

이 과제에서의 반 스텝

자세각	각속도가 상수라 반 스텝 값을 바로 더한다
위치	스텝 시작 속도로 반 스텝 예측한다
속도	그 자리 · 그 자세에서 다시 만든다

계산량은 좌표변환 2회 → 4회로 늘지만, 0.1 s 에서 오차가 1.36 m → 1.36 cm 로 100 배 줄었다.

22

발표 대본

구현 · 약 60초

위치와 속도를 구했으면 이제 0.1초를 전진시켜야 합니다.

왼쪽 위가 제일 단순한 방법입니다. 지금 속도로 0.1초 곱해서 더한다. 오일러법이라고 합니다.

직진이면 아무 문제가 없습니다. 그런데 선화 중이면 0.1초 동안에도 기수가 계속 돌고 있는데, 출발할 때의 방향으로 쪽 밀어 버리니까 굽은 길을 직선 토막으로 근사하게 됩니다. 늘 바깥쪽으로 밀립니다. 그림 오른쪽 끝에 "벌어짐" 이라고 표시한 부분입니다.

왼쪽 아래가 중점법입니다. 일단 반 걸음만 가봅니다. 회색 점선이 그겁니다. 거기서 방향을 다시 재고, 그 방향으로 한 걸음을 갑니다. 반 걸음 간 자리의 기울기가 구간 평균에 훨씬 가깝습니다.

오른쪽이 식과 오차 차수입니다. 오일러법은 국소 오차가 델타티 제곱, 전역 오차가 델타티에 비례합니다. 1차입니다. 중점법은 국소가 세제곱, 전역이 제곱입니다. 2차입니다.

2차라는 말은 간격을 절반으로 줄이면 오차가 4분의 1이 된다는 뜻입니다. 이걸 뒤에서 숫자로 확인하겠습니다.

계산량은 좌표변환 두 번에서 네 번으로 늘어납니다. 대신 0.1초에서 오차가 1.36미터에서 1.36센티미터로, 100 배 줄었습니다. 그 정도면 치를 만한 값이라고 봤습니다.

02. 표적 궤적 모의 구현

코드 ④ — 한 스텝 전진 (중점법)

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발해)

```

halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세 — 각속도가 구간 내 상수라 바로 더한다
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리, 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...

```

②를 빼먹었을 때 생긴 일

처음에는 자세만 반 스텝 돌리고 위치는 그대로 두었다. 그러면 스텝 시작 지점의 위경도로 NED를 잡게 되어, 표적은 앞으로 나아갔는데 "북쪽"의 방향은 뒤에 남는다.

그 차이가 매 스텝 $d'/2R$ 만큼 고도로 쌓인다.

알고도 남겨 둔 한계 — 짐벌락

오일러각을 직접 적분하므로 Pitch가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐 Yaw 기동이 궤적을 바꾸지 못한다.

사원수로 바꾸면 풀리지만 과제 범위를 넘는다고 보고 코드에 주석으로 명시했다.

23

발표 대본

코드 · 약 70초

중점법이 코드로는 이렇게 생겼습니다. 네 덩어리입니다.

- ①번. 자세각을 반 스텝 돌립니다. 각속도가 구간 안에서 상수라 그냥 더하면 됩니다.
- ②번. 반 스텝 뒤의 위치를 구합니다. 스텝 시작 속도로 예측합니다.
- ③번. 그 자리, 그 자세에서 속도를 다시 구합니다. 이게 이 스텝의 대표 속도입니다.
- ④번. 그 속도로 한 스텝을 옮깁니다.

오른쪽 위 카드가 ②번 이야기인데, 여기가 제가 처음에 빼먹었던 부분입니다.

원래는 자세만 반 스텝 돌리고 위치는 그대로 두고 속도를 만들었습니다. 그렇게 해도 겉보기에는 잘 돌아갑니다. 그런데 수평으로 날린 표적의 고도가 스텝마다 아주 조금씩 올라갔습니다.

이유가 이겁니다. 스텝 시작 지점의 위경도로 NED 축을 세우면, 표적은 이미 앞으로 나아갔는데 "북쪽"의 방향은 뒤에 남아 있는 셈이 됩니다. 앞 장에서 본 d 제공 나누기 $2R$ 이 바로 이 항입니다. 위치까지 반 스텝 옮기고 나서 이 항이 사라졌습니다.

오른쪽 아래가 알고도 남겨 둔 한계입니다. 오일러각을 직접 적분하니까 Pitch가 플러스마이너스 90도 근처로 가면 Yaw 축이 속도 방향과 겹쳐서 Yaw 기동이 궤적을 못 바꿉니다. 짐벌락입니다. 사원수를 쓰면 풀리는데 과제 범위를 넘는다고 보고 주석으로 명시했습니다.

02. 표적 궤적 모의 구현

한 스텝 알고리즘과 계산 비용

f_Tgt_StepSim — 한 스텝에 벌어지는 일

for 각 객체 (플랫폼 1 + 표적 N)

① 현재 시각에 걸린 가동을 찾는다
`rate = f_Tgt_GetAttRate(target, curTime)`

② 자세각을 반 스텝 돌린다
`attMid = att + rate * (dt / 2)`

③ 위치를 반 스텝 예측한다
`velStart = GetVelEcef(att, lla)`
`posMid = pos + velStart * (dt / 2)`
`llaMid = Ecef_To_Lla(posMid)`

④ 그 자리 · 그 자세의 속도로 한 스텝 간다
`velMid = GetVelEcef(attMid, llaMid)`
`posNew = pos + velMid * dt`

⑤ 출력용 LLA 와 다음 속도를 만든다
`llaNew = Ecef_To_Lla(posNew)`
`velNew = GetVelEcef(attNew, llaNew)`

한 스텝의 비용

좌표변환 호출 4 회

GetVelEcef 2 + Ecef_To_Lla 2

행렬 곱셈 8 회

제공 라이브러리 Matrix_Product2

삼각함수 약 20 회

DCM 원소 계산

분기 · 나눗셈 기동 조회 1 회

속력으로 나누는 곳은 여기뿐

실측 성능

601 표본 × 객체 3 4.8 ms

한 스텝 · 한 객체 약 2.7 us

6001 표본 × 객체 11 0.5 s 미만

동적 할당 없음 · 스텝 간 상태 의존만 있음

④ 가 이 과제의 핵심이다

③ 에서 위치까지 반 스텝 옮기지 않으면

수평 비행 고도가 스텝마다 올라간다.

24

발표 대본

구현 · 약 55초

왼쪽이 지금까지 이야기한 것을 의사코드로 적은 것입니다. 플랫폼 하나와 표적 N 개에 대해 이 다섯 단계가 됩니다. 색이 옅은 줄이 주석이고, 실제로 하는 일은 대입문 몇 줄입니다.

오른쪽 위가 한 스텝의 비용입니다. 좌표변환 호출이 네 번, 그 안에서 행렬 곱셈이 여덟 번, 삼각함수가 스무 번쯤입니다. 나눗셈이 들어가는 곳은 기동 조회 한 군데뿐입니다.

그 아래가 실측 성능입니다. 601 표본에 객체 3개면 4.8밀리초입니다. 한 스텝 한 객체가 2.7마이크로초쯤 됩니다. 표적을 10개로 늘리고 간격을 0.01초로 줄여도 0.5초 안에 끝납니다.

이 정도면 화면에서 값을 고칠 때마다 다시 돌려도 사용자가 기다림을 못 느낍니다. 실제로 UI 는 그렇게 동작합니다.

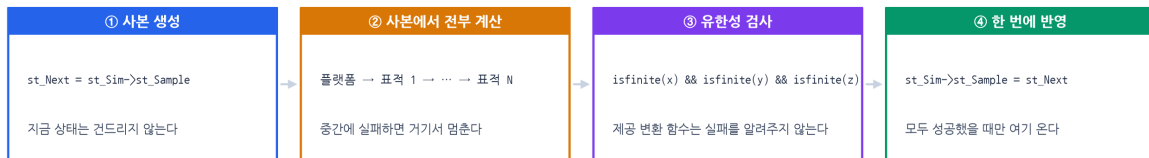
맨 아래 빨간 상자가 다시 한번 ④번 강조입니다.

02. 표적 궤적 모의 구현

계산이 깨져도 상태는 지킨다

제공받은 변환 함수는 실패를 알려 주지 않습니다. 그래서 결과의 유한성을 직접 확인합니다.

계산이 깨져도 상태는 지킨다



그래서 생기지 않는 상태

- 표적 1 은 새 시각인데 표적 2 는 옛 시각인 표본
- 위치는 갱신됐는데 속도는 갱신 전인 객체

오류 코드 NaN 이 섞인 채 다음 스텝으로 넘어가 전체가 오염되는 것

TGT_ERR_TARGET_NUM
표적 수 범위 밖
TGT_ERR_MANEUVER_OVERLAP
기동 구간 겹침

TGT_ERR_MANEUVER_NUM
기동 수 범위 밖
TGT_ERR_COORD
변환 결과가 비유한

TGT_ERR_TIME
시간 · 간격 · 기동 구간 이상
TGT_ERR_SIM_STATE
호출 사이 설정이 바뀜

TGT_ERR_SPEED
속력 0 이하 또는 비유한
TGT_ERR_SIM_END
마지막 스텝까지 진행 완료

25

발표 대본

구현 · 약 55초

마지막으로 방어적 구현입니다.

위 네 칸이 한 스텝의 처리 순서입니다. 먼저 지금 상태를 사본으로 씁니다. 사본에서 전부 계산합니다. 결과가 유한한지 확인합니다. 다 통과했을 때만 한 번에 반영합니다.

왜 이렇게 했냐면, 제공받은 변환 함수들이 값만 돌려줄 뿐 실패를 알려주지 않기 때문입니다. 0으로 나누거나 정의역을 벗어나도 NaN 이나 무한대를 그냥 리턴합니다. 그래서 결과가 유한한지를 제가 직접 봅니다.

가운데 분홍 상자가 이렇게 해서 생기지 않는 상태들입니다. 표적 1은 새 시각인데 표적 2는 옛 시각인 표본, 위치는 갱신됐는데 속도는 안 된 객체, NaN 이 섞인 채 다음 스텝으로 넘어가서 전체가 오염되는 것.

이런 건 한 번 생기면 어디서부터 틀렸는지 찾기가 굉장히 어렵습니다. 그래서 아예 못 생기게 막았습니다.

아래가 오류 코드 목록입니다. 맨 오른쪽 초록색 하나만 오류가 아니라 정상 종료입니다.

03 시뮬레이션 결과 및 검증

Results and Verification

- 명세 조건 그대로 60 초를 돌린 결과
- 해석해 대조 · 불변량 보존 · 수렴 차수 — 세 가지 검증
- 기동을 엮은 시연과 운용 화면

26

발표 대본

장절 · 약 25초

3장, 결과입니다.

명세 조건 그대로 돌린 궤적을 보여 드리고, 그게 맞다는 것을 세 가지 방법으로 확인하겠습니다. 그림이 그럴듯하게 나왔다고 맞는 건 아니니까요.

03. 시뮬레이션 결과 및 검증

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60s · 간격 0.1s

스텝 600 회, 표본 601 개.
 $t=0$ 과 $t=60$ 을 모두 포함한다.
 시각은 누적이 아니라 스텝 번호 $\times$ 간격으로
 매겨 반올림 오차가 쌓이지 않게 했다.

검증하기 좋은 조건이다

기동이 없으므로 답을 미리 알 수 있다.
 대함은 정서진 $\rightarrow$ 위도 불변,
 대공은 정남진 $\rightarrow$ 경도 불변,
 둘 다 수평 $\rightarrow$ 고도 불변.

플랫폼 속력이 0 인 점

각속도 계산에 속력이 분모로 들어가지만,
 그 계산은 기동 조회 안에만 있다.
 플랫폼은 기동이 없어 그 자리를 지나가지 않는다.

27

발표 대본

결과 · 약 50초

과제 문서 2항의 값을 그대로 넣었습니다.

플랫폼은 위도 32도, 경도 126도, 고도 0, 속력 0 입니다. 움직이지 않습니다.

대함 표적은 위도 32.125, 경도 126.03, 고도 0, 초속 30미터, Yaw 270도. 270도면 서쪽입니다.

대공 표적은 위도 32.12, 경도 126.0, 고도 300미터, 초속 200미터, Yaw 180도. 남쪽입니다.

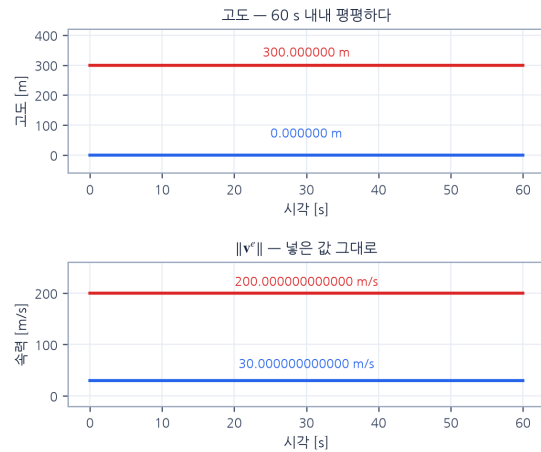
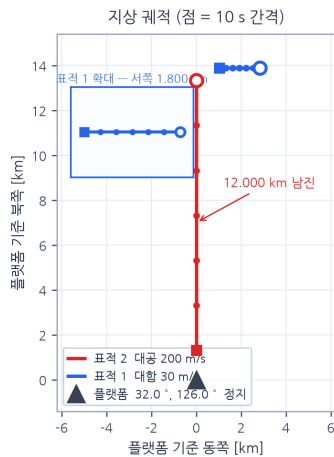
60초에 0.1초 간격이니까 스텝이 600번, 표본이 601개입니다. 시각은 누적해서 더하지 않고 스텝 번호에 간격을 곱해서 매겼습니다. 0.1을 600번 더하면 반올림 오차가 쌓이는데, 곱셈은 한 번이라 안 쌓입니다.

가운데 카드가 중요합니다. 이 조건은 검증하기 아주 좋은 조건입니다. 기동이 없으니까 답을 미리 알 수 있습니다. 대함은 정서쪽으로만 가니까 위도가 변하면 안 되고, 대공은 정남쪽으로만 가니까 경도가 변하면 안 되고, 둘 다 수평이니까 고도가 변하면 안 됩니다.

03. 시뮬레이션 결과 및 검증

결과 ① — 60 초 궤적

플랫폼을 원점으로 잡은 동-북 평면입니다. 점 하나가 10 초입니다.



28

발표 대본

결과 · 약 55초

결과입니다. 플랫폼을 원점으로 잡은 동-북 평면이고, 점 하나가 10초입니다.

빨간색 대공 표적이 정남쪽으로 12킬로미터를 내려갔습니다. 초속 200미터 곱하기 60초니까 12킬로미터가 맞습니다.

파란색 대함 표적은 너무 작아서 왼쪽 위에 확대해서 넣었습니다. 서쪽으로 1.8킬로미터입니다.

둘 다 완벽한 직선입니다. 기동을 안 걸었으니 그래야 합니다.

오른쪽 위가 고도입니다. 60초 내내 평평합니다. 대공은 300.000000미터, 대함은 0.000000미터입니다.

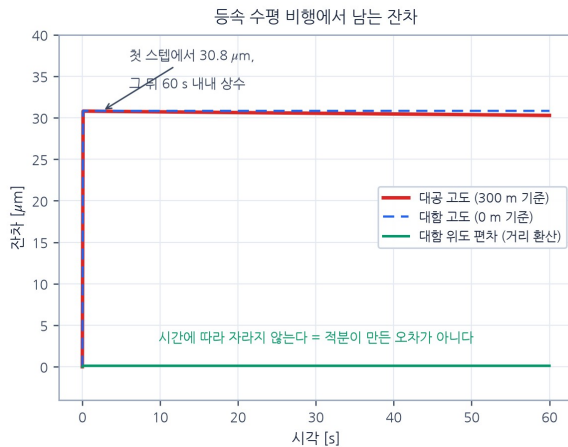
오른쪽 아래가 ECEF 속도의 크기입니다. 소수점 열두 자리까지 넣은 값 그대로입니다. 앞에서 회전행렬이 크기를 안 바꾼다고 말씀드린 그 성질이 그대로 나온 겁니다.

여기까지가 눈으로 보는 확인이고, 다음 세 장이 숫자로 하는 확인입니다.

03. 시뮬레이션 결과 및 검증

결과 ② — 불변량 잔차

변하면 안 되는 값이 실제로 얼마나 변했는지, 마이크로미터 단위까지 봤습니다.



불변량별 최대 잔차 (601 표본)

대공 고도	30.8 μm
제공 코드의 ECEF→LLA 5회 반복 근사	
대함 고도	30.8 μm
같은 원인 — 두 표적이 같은 값	
대함 위도	0.11 μm
정서진 — 위도가 변하면 안 된다	
대공 경도	0.00 μm
정남진 — 경도가 변하면 안 된다	
대공 속력	0 m/s (완전 일치)
회전행렬이 크기를 보존한다	
대함 속력	0 m/s (완전 일치)
601 표본 모두 비트까지 같다	

잔차의 출처는 적분기가 아니라 제공받은 좌표변환의 반복 근사다.
Bowring 폐형식으로 바꾸면 사라지지만, 과제 조건이 "제공 코드 사용"이라 그대로 두었다.

29

발표 대본

결과 · 약 65초

변하면 안 되는 값이 실제로 얼마나 변했는지를 마이크로미터 단위까지 봤습니다.

왼쪽 그래프에서 세로축 단위가 마이크로미터입니다. 100만분의 1미터입니다.

빨간색이 대공 고도, 파란 점선이 대함 고도인데 둘이 겹쳐 있습니다. 첫 스텝에서 30.8마이크로미터 올랐다가 그 뒤 60초 내내 그대로입니다. 초록색 대함 위도 편차는 사실상 0 입니다.

여기서 읽을 것은 값의 크기가 아니라 모양입니다. 시간에 따라 자라지 않습니다. 계단 하나 올라간 다음 평평합니다.

적분이 만든 오차라면 시간에 비례해서 커져야 합니다. 그런데 안 커집니다. 그러면 적분 때문이 아니라는 뜻입니다.

원인을 찾아보니 제공받은 ECEF 에서 LLA 로 가는 함수였습니다. 그 함수가 반복법으로 다섯 번 돌아서 근사값을 냅니다. $t = 0$ 일 때는 제가 넣은 LLA 를 그대로 쓰니까 오차가 0 이고, 첫 스텝부터는 변환한 값을 쓰니까 그 근사의 몫이 상수로 얹힙니다.

오른쪽 표가 불변량별 최대 잔차입니다. 고도가 30.8마이크로미터, 위도 편차가 0.11마이크로미터, 경도는 완전히 0, 속력은 두 표적 모두 0 입니다. 여기서 0 은 반올림해서 0 이 아니라 비트까지 같다는 뜻입니다.

Bowring 폐형식으로 바꾸면 고도 잔차도 사라집니다. 실제로 그 판을 따로 만들어 봤는데, 과제 조건이 "제공한 좌표 변환 코드 이용" 이라서 본체는 원본을 쓰고 개선본은 ref 폴더에 보관해 뒀습니다.

03. 시뮬레이션 결과 및 검증

결과 ③ — 검증 종합

세 가지 방법으로 검증했다

① 해석해와 대조	이론값 · 기대값	시뮬레이션 결과	잔차
대함 이동거리	$30 \times 60 = 1,800 \text{ m}$	1800.00000 m	+1.25e+08 m
대공 이동거리	$200 \times 60 = 12,000 \text{ m}$	12000.00000 m	+1.66e+07 m
② 불변량 보존	이론값 · 기대값	시뮬레이션 결과	잔차
대공 고도	300 m 유지	300.000000 ~ 300.000031 m	30.8 μm
대공 속력	200 m/s 유지	200.000000000000 m/s	0.0e+00
대함 위도	32.125 ° 유지 (정서진)	32.125000000000 °	0.1 μm
대공 경도	126.0 ° 유지 (정남진)	126.000000000000 °	0
③ 수렴 차수	이론값 · 기대값	시뮬레이션 결과	잔차
Δt 를 1/2 로	오차 1/4 (2차)	1.357 $\rightarrow$ 0.339 $\rightarrow$ 0.085 m	기울기 2.00
$\Delta t = 0.1 \text{ s}$	— (설계값 확인)	60 s 뒤 위치오차 1.36 cm	분해능 대비 무시

이동거리는 601개 표본 사이 거리를 전부 더해 켜 것이다 — 직선거리가 아니라 실제로 지나간 길이.
12 km 를 날아 잔차가 0.05 mm 면 적분기의 오차가 아니라 double 형의 표현 한계에 가깝다.

30

발표 대본

결과 · 약 60초

검증을 세 가지 방법으로 나눠서 했습니다.

첫째, 해석해와 대조입니다. 답을 손으로 계산할 수 있는 항목을 골랐습니다. 대함은 초속 30미터 곱하기 60초 해서 1800미터, 대공은 12000미터여야 합니다. 시뮬레이션 결과가 각각 1.25에 10의 마이너스 8제곱 미터, 1.66에 10의 마이너스 7제곱 미터 차이입니다. 12킬로미터를 날아서 0.0002밀리미터 차이입니다.

이 거리는 601개 점 사이의 거리를 전부 더해서 켜 것입니다. 시작점과 끝점 사이 직선거리가 아니라 실제로 지나간 길이입니다.

둘째가 방금 본 불변량 보존입니다.

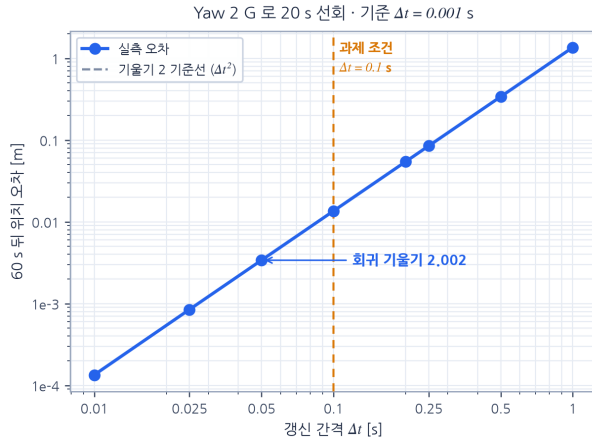
셋째가 수렴 차수인데, 이게 세 방법 중에 제일 강한 증거라고 생각합니다. 다음 장에서 자세히 보겠습니다.

왜 세 가지로 나뉘냐면, 각각이 다른 종류의 실수를 잡기 때문입니다. 첫째는 스케일이 틀린 경우를 잡고, 둘째는 방향이 틀린 경우를 잡고, 셋째는 적분 방법이 틀린 경우를 잡습니다.

03. 시뮬레이션 결과 및 검증

결과 ④ — 갱신 간격에 대한 수렴

값 하나가 작은 것보다, 간격을 바꿨을 때 정해진 차수로 줄어드는 것이 더 강한 증거입니다.



읽는 법

두 선이 나란하다

간격을 절반으로 줄이면 오차가 1/4로 줄어든다.
중점법이 설계대로 2차 정확도로 돌고 있다는 뜻.

 $\Delta t = 0.1$ s 에서 1.36 cm

20 s 동안 2 G로 꺾는 내내 쌓인 값이다.
레이다 거리 분해능(수 m)에 비하면 없는 값.

직진이면 더 작다

기동이 없으면 오차가 10 nm 수준 —
사실상 부동소수점 오차만 남는다.

차수가 확인의 핵심

값 하나가 작은 것보다, 간격을 바꿨을 때
정해진 기울기로 줄어드는 것이 구현이 옳다는 증거다.

발표 대본

결과 · 약 65초

직진은 틀릴 수가 없으니 가장 많이 휘는 경우로 확인했습니다.

대공 표적에 2G로 20초 선회를 걸어 놓고, 갱신 간격만 1초부터 0.01초까지 바꿔 가며 60초를 돌렸습니다. 기준은 0.001초로 돌린 결과입니다.

왼쪽 그래프는 양쪽 축이 모두 로그입니다. 파란 실선이 실측 오차고, 회색 점선이 "간격의 제곱에 비례하는 선"입니다. 둘이 나란합니다.

나란하다는 건 간격을 절반으로 줄이면 오차가 4분의 1이 된다는 뜻입니다. 실제로 회귀로 기울기를 재보면 2.002가 나옵니다. 중점법이 설계한 대로 2차 정확도로 돌고 있다는 증거입니다.

주황 세로선이 과제 조건인 0.1초 자리입니다. 거기서 오차가 1.36센티미터입니다. 20초 동안 2G로 꺾는 내내 쌓인 게 1.4센티미터입니다. 레이더 거리 분해능이 보통 수 미터 단위인 걸 생각하면 없는 값이라고 봐도 됩니다.

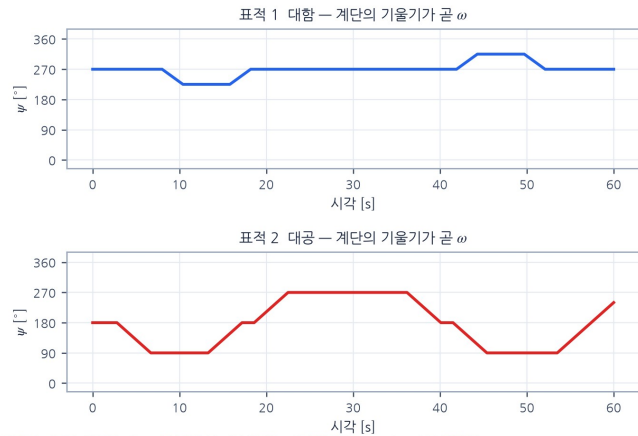
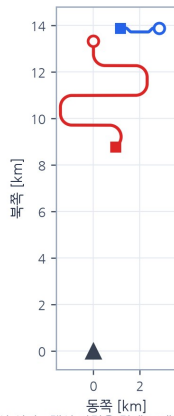
오른쪽 마지막 칸이 제가 이 장에서 제일 하고 싶은 말입니다. 값 하나가 작은 건 사실 별 의미가 없습니다. 우연히 작을 수도 있으니까요. 그런데 간격을 바꿨을 때 정해진 기울기로 줄어들면, 그건 구현이 의도한 방법을 정확히 따르고 있다는 뜻입니다.

03. 시뮬레이션 결과 및 검증

결과 ⑤ — 기동을 없으면

초기값 · 시뮬레이션 시간 · 갱신 간격은 명세 그대로, 기동표 10줄만 추가했습니다.

기동 10줄을 없은 60 s 궤적



초기값 · 시뮬레이션 시간 · 갱신 간격은 명세 그대로. 기동표 10줄만 추가했다. 회전 지점이 좌표로 주어져, Core 를 돌려 그 좌표에 닿는 시각을 찾아 start / end 로 옮겼다.

32

발표 대본

결과 · 약 50초

마지막으로 기동을 없은 결과입니다.

초기값도, 시뮬레이션 시간도, 갱신 간격도 명세 그대로입니다. 기동표 10줄만 추가했습니다.

빨간색 대공 표적이 남쪽으로 내려가다가 동쪽으로 꺾고, 다시 남쪽, 다시 서쪽으로 훑고 내려갑니다. 파란 대함은 서쪽으로 가다가 한 번 비켜 갑니다.

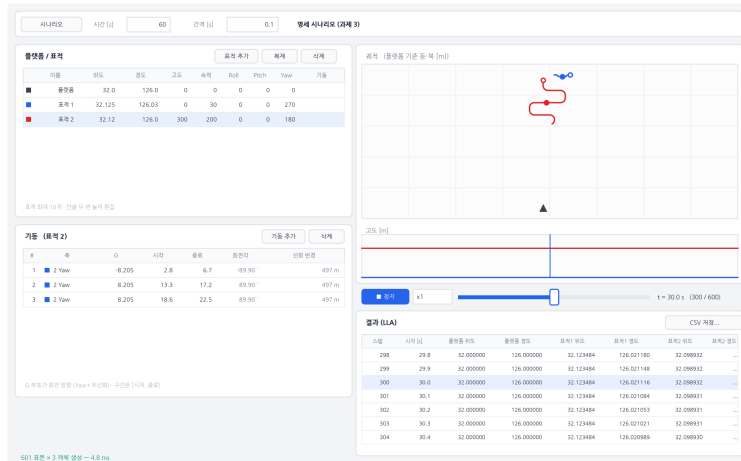
오른쪽 두 그림이 기수 방위각입니다. 계단처럼 생겼는데, 평평한 데가 직진이고 기울어진 데가 기동입니다. 기울기가 곧 각속도이고, 그 값이 1장에서 본 n 곱하기 g 나누기 V 입니다.

아래 설명에 적어 뒀는데, 회전 지점이 좌표로 주어져서 시각으로 바꾸는 작업을 따로 했습니다. Core 를 실제로 돌려서 그 좌표에 닿는 시각을 찾았고, 회전하는 동안에도 표적이 계속 나아가니까 선회 반경만큼 미리 꺾어야 지정한 자리에서 수평이 됩니다.

03. 시뮬레이션 결과 및 검증

TargetSim — 값을 바꿔 가며 확인하는 화면

표에서 숫자를 고치면 자동으로 다시 돌고, 그림에서 시작점과 기수를 끌 수 있습니다.



고치면 곧바로 다시 돈다

표의 칸을 고치면 0.25 초 뒤 자동으로 다시 계산한다. 601 표본 × 3 객체가 4.8 ms 라 기다릴 일이 없다.

그림에서 끌어서 배치

시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 반영된다. 회전각과 선회 반경은 G 로부터 계산해 보여 준다.

결과는 표 · CSV · 그림

시각별 LLA 를 표로 보고, 슬라이더로 되감고, CSV 로 내보낸다. 시나리오는 텍스트 파일로 저장해 다시 불러올 수 있다.

발표 대본

결과 · 약 50초

값을 코드에 박아 놓고 다시 컴파일하는 건 불편해서 화면을 붙였습니다.

왼쪽 위 표에서 표적 초기값을 고치면 0.25초 뒤에 자동으로 다시 계산합니다. 4.8밀리초면 끝나니까 기다릴 일이 없습니다. 왼쪽 아래가 기동표인데, 회전각과 선회 반경은 입력한 G 와 시간으로 계산해서 같이 보여 줍니다. 앞에서 본 그 식입니다.

오른쪽 위가 궤적입니다. 시작점 표식을 마우스로 끌면 위도와 경도가 바뀌고, 기수 손잡이를 끌면 Yaw 가 바뀝니다. 표에 숫자로 넣는 것보다 시나리오를 잡을 때 훨씬 빠릅니다.

아래가 결과 표인데 시각마다의 LLA 가 그대로 나옵니다. 슬라이더로 되감아 볼 수 있고, CSV 로 내보낼 수 있습니다. 오늘 보여 드린 숫자들이 전부 여기서 나온 CSV 입니다.

03. 시뮬레이션 결과 및 검증

정리

요구사항과 결과를 한 장에 놓고 보겠습니다.

과제 요구사항	구현	근거
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_heading	ST_TargetInit, 방향은 자세각이 담당	검증 통과
기동 = G · 축 · 시작 · 종료	ST_TargetManeuver 4 필드, 구간 겹침 검사	검증 통과
ECEF 에서 상태 갱신	위치 · 속도 ECEF, 출력용 LLA 병행 보관	고도 잔차 30 μm
V_heading → Vx, Vy, Vz (추가 고민)	동체 → NED → ECEF, 회전 2 회	속력 완전 보존
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본	무수정
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 · 고도 · 방위 그림	601 표본 속력

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 — 사원수로 전환하면 해소
- 힘을 풀지 않는 운동학 모델 — 탄도 표적에는 부적합
- 고도 잔차 30.8 μm 는 제공 코드의 ECEF→LLA 반복 근사

이어 붙일 수 있는 것

- 궤적에 측정 잡음을 없으면 그대로 추적 필터 입력
- Part 2 의 좌표변환을 붙이면 안테나 기준 R · Az · El
- 표본 생성기를 바꾸면 RCS · 도플러까지 확장 가능

34

발표 대본

정리 · 약 65초

요구사항과 결과를 한 장에 놓고 보겠습니다.

표적 10개와 기동 30개는 고정 배열로 상한을 두고, 범위를 넘으면 오류 코드로 거부했습니다.

위치는 LLA, 속도는 V_heading 하나로 받았고 방향은 자세각이 담당합니다.

기동은 축, G, 시작, 종료 네 필드로 정의하고 구간 겹침까지 검사합니다.

상태 갱신은 ECEF 에서 하고, 출력용 LLA 를 같이 보관합니다. 고도 잔차 30마이크로미터는 좌표변환 근사의 몫입니다.

추가 고민 항목이었던 속도 변환은 동체에서 NED, NED 에서 ECEF 로 두 번 회전했고, 속력이 완전히 보존되는 것을 601개 표본으로 확인했습니다.

좌표변환 코드는 제공받은 것을 무수정으로 썼습니다.

결과는 표와 CSV, 그리고 궤적, 고도, 방위 그림으로 냈습니다.

아래 왼쪽이 남겨 둔 한계입니다. 짐벌락, 힘을 풀지 않는 운동학 모델, 그리고 고도 잔차. 셋 다 원인을 알고 남겨 둔 것이고 각각 해결 방법도 적어 뒀습니다.

오른쪽이 이어 붙일 수 있는 것입니다. 이 궤적에 측정 잡음만 없으면 바로 추적 필터 입력이 됩니다. Part 2 에서 만든 좌표변환을 붙이면 안테나 기준 거리와 방위각, 고각이 나옵니다. 다음 과제로 이어지는 부분입니다.

감사합니다

질문 주시면 답변드리겠습니다

35

발표 대본

마무리 · 약 15초

이상입니다. 감사합니다.

질문 있으시면 답변드리겠습니다.